PureMilk

# End-to-End Azure Data Engineering Project Demonstration Report

*Smart Dairy Milk Silo Data Pipeline*

*by*

***Dhilipkumar***

## 1. Executive Summary

The PureMilk project demonstrates an end-to-end data engineering pipeline for milk-silo and laboratory sensor records. A Python generator produces sample events and publishes them to Azure Event Hubs. An Event Hub consumer writes the incoming events as JSON files to Azure Blob Storage. Azure Data Factory orchestrates incremental processing and a Mapping Data Flow cleans, validates, and separates accepted and rejected records. Valid processed records are loaded into Azure SQL Database, the watermark is updated after successful processing, and eligible source files are cleaned up.

## 2. Project Objectives

- Generate representative milk-silo sensor data and stream it through Azure Event Hubs.
- Land incoming events as files in Azure Blob Storage.
- Use ADF Mapping Data Flow to clean and validate records, remove duplicates, and route invalid records to a rejected output.
- Load valid data into Azure SQL Database.
- Use a watermark to control incremental processing and avoid reprocessing previously handled data.
- Run cleanup only after the required processing, SQL load, and watermark operations succeed.
- Demonstrate the complete pipeline and capture evidence for each stage.

## 3. Architecture and Data Flow

**1. Source data -** Milk-silo/laboratory data is represented by generated sensor records.
**2. Python Data Generator -**Creates sample events and sends them to Azure Event Hubs.
**3. Azure Event Hubs -** Receives the streaming events.
**4. Event Hub Consumer -** Consumes events and writes JSON files to Blob Storage.
**5. Blob Storage – Landing/Raw -** Stores the source files for downstream processing.
**6. Azure Data Factory -** Orchestrates the pipeline and incremental processing.
**7. Mapping Data Flow -** Cleans fields, validates values, handles duplicates, and splits valid and invalid records.
**8. Processed / Rejected storage -** Stores accepted output separately from rejected records.
**9. Azure SQL Database** Receives the processed records and maintains the watermark table.
**10. Watermark and cleanup -** Updates the checkpoint after successful loading, then removes only eligible source/temporary files.

# 4. Demonstration Procedure and Evidence

## 4.1 Python Data Generator

Run the generator and show that it creates milk-silo events with fields such as sample ID, silo ID, timestamp, temperature, pH, acidity, fat, SNF, density, protein, and relevant quality indicators.

- Screenshot:

[5]:
```
with producer:
    while True:
        batch_time = datetime.now()
        batch = generate_milksilo_batch(batch_time, n=20)
        send_data(producer, batch)
        print(f"{batch_time} Milk Silo data Generated : {batch['batch_id']} ({len(batch['samples'])} samples)")

        now = datetime.now()
        next_minute = (now + timedelta(minutes=1))
        sleep_seconds = (next_minute - now).total_seconds()
        time.sleep(sleep_seconds)
```

```
2026-10-14 14:49:26.959196 Milk Silo data Generated : 20261014_1449 (20 samples)
2026-10-14 14:50:30.621246 Milk Silo data Generated : 20261014_1450 (20 samples)
2026-10-14 14:51:31.530644 Milk Silo data Generated : 20261014_1451 (20 samples)
2026-10-14 14:52:31.623118 Milk Silo data Generated : 20261014_1452 (20 samples)
2026-10-14 14:53:31.726378 Milk Silo data Generated : 20261014_1453 (20 samples)
2026-10-14 14:54:31.822653 Milk Silo data Generated : 20261014_1454 (20 samples)
```

## 4.2 Azure Event Hubs

Show the Event Hub namespace and event hub used by the generator. Explain that the generator publishes the generated events to the configured hub.

- Screenshot:

```
from azure.eventhub import EventHubConsumerClient
from azure.eventhub.extensions.checkpointstoreblob import BlobCheckpointStore
from azure.storage.blob import BlobServiceClient

import json

# ---- CONFIG ----
# Same namespace-level policy as the producer, pointed at your event hub.
eh_con_str_con = "Endpoint=sb://ehorder.servicebus.windows.net/;SharedAccessKeyName=consumer-listen;SharedAccessKey=tRW9/TWVDxGNrYWPlDXane1qMMsESuhvs+AEh
eh_name = "ehstoreawebdilip"

# Same shared storage account you already used for the web/store project.
storage_conn = "DefaultEndpointsProtocol=https;AccountName=ivyde2026;AccountKey=B276MWkKrQKSiOvJq2KTdy5NGDByvdhmLdnqervh2uWP9td5rgpoyu4MmAore09V8MX45p8/0

land_container = "ivydeaug"
user_folder = "Dhilip_Projects"               # your existing personal folder in ivydeaug
landing_prefix = f"{user_folder}/Smart Dairy Milk Silo/bronze"

# Use YOUR OWN checkpoint container (already created for you) instead of the
# shared "eh-checkpoint" one, so your consumer offsets don't collide with anyone else's.
checkpoint_container = "eh-checkpoint-dilip"

blob_service = BlobServiceClient.from_connection_string(storage_conn)
landing_cont = blob_service.get_container_client(land_container)
checkpoint_store = BlobCheckpointStore.from_connection_string(storage_conn, checkpoint_container)
```

## 4.3 Event Hub Consumer

Run or show the consumer. Explain how it reads events from Event Hubs and writes them as JSON files to Azure Blob Storage.

- Screenshot:

```
consumer = EventHubConsumerClient.from_connection_string(
    conn_str=eh_con_str_con, consumer_group='$Default',
    eventhub_name=eh_name, checkpoint_store=checkpoint_store
)

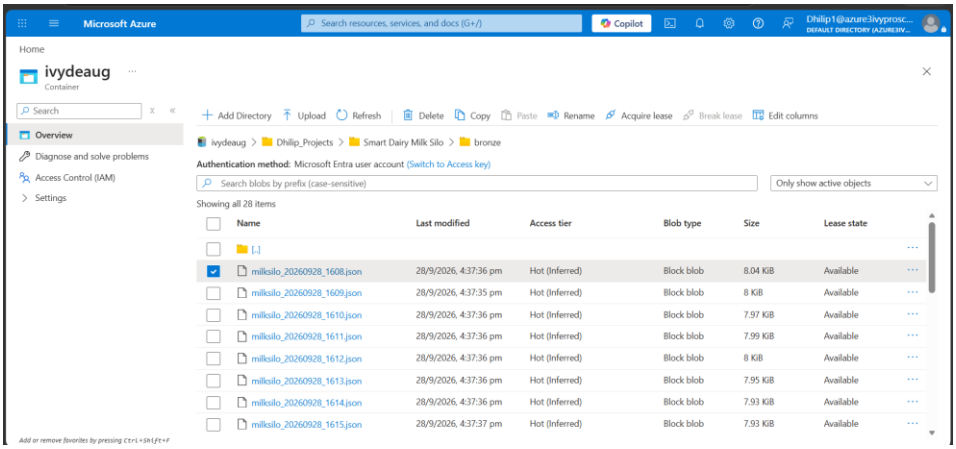
with consumer:
    consumer.receive(on_event, starting_position='-1')
```

```
Milk Silo file created : Dhilip_Projects/Smart Dairy Milk Silo/bronze/milksilo_20261014_1450.json (20 samples)
Milk Silo file created : Dhilip_Projects/Smart Dairy Milk Silo/bronze/milksilo_20261014_1449.json (20 samples)
Milk Silo file created : Dhilip_Projects/Smart Dairy Milk Silo/bronze/milksilo_20261014_1452.json (20 samples)
Milk Silo file created : Dhilip_Projects/Smart Dairy Milk Silo/bronze/milksilo_20261014_1451.json (20 samples)
Milk Silo file created : Dhilip_Projects/Smart Dairy Milk Silo/bronze/milksilo_20261014_1454.json (20 samples)
Milk Silo file created : Dhilip_Projects/Smart Dairy Milk Silo/bronze/milksilo_20261014_1453.json (20 samples)
```

## 4.4 Blob Storage – Landing/Raw

Open the configured container and Landing/Raw path. Show at least one JSON file and inspect its structure.

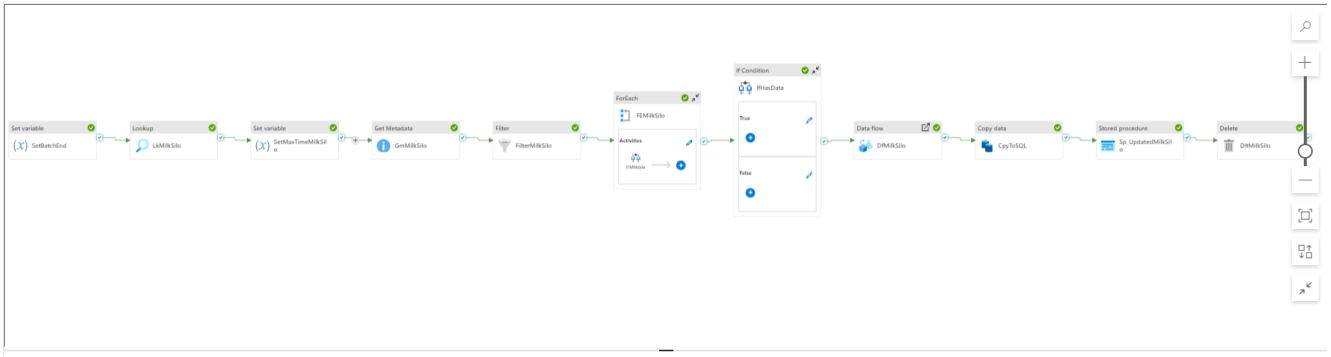
Screenshot:

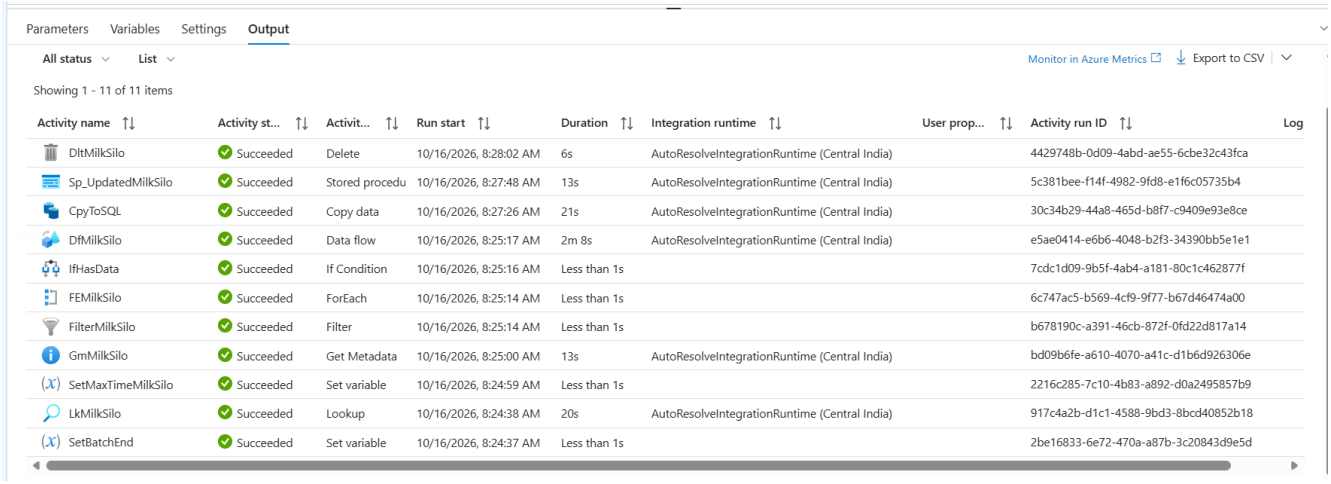



## 4.5 Azure Data Factory Pipeline

Open the ADF pipeline and explain the control flow, source selection, incremental/watermark logic, Data Flow execution, SQL load, watermark update, and cleanup dependencies. Trigger a run and show its actual status.

- Screenshot:



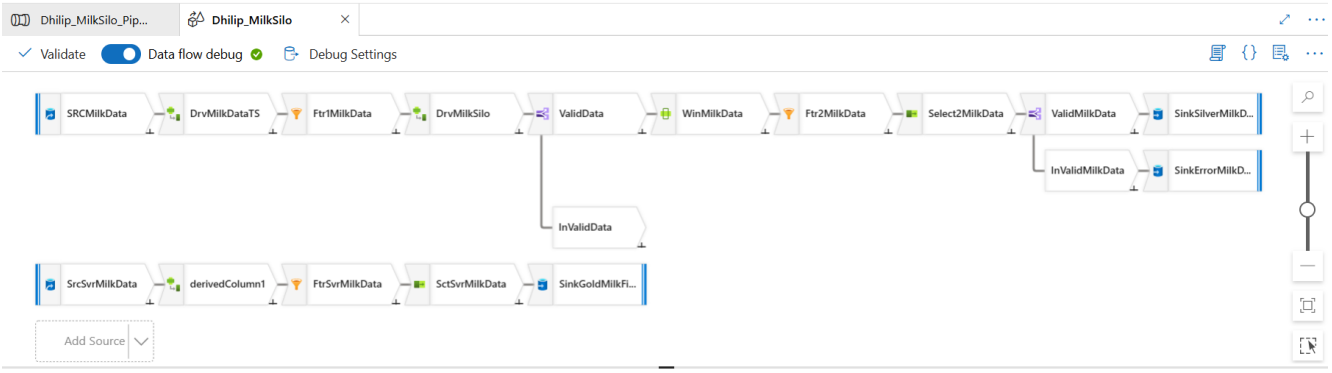



| Activity name | Activity st... | Activit... | Run start | Duration | Integration runtime | User prop... | Activity run ID | Log |
|---|---|---|---|---|---|---|---|---|
| DltMilkSilo | Succeeded | Delete | 10/16/2026, 8:28:02 AM | 6s | AutoResolveIntegrationRuntime (Central India) | | 4429748b-0d09-4abd-ae55-6cbe32c43fca | |
| Sp_UpdatedMilkSilo | Succeeded | Stored procedu | 10/16/2026, 8:27:48 AM | 13s | AutoResolveIntegrationRuntime (Central India) | | 5c381bee-f14f-4982-9fd8-e1f6c05735b4 | |
| CpyToSQL | Succeeded | Copy data | 10/16/2026, 8:27:26 AM | 21s | AutoResolveIntegrationRuntime (Central India) | | 30c34b29-44a8-465d-b8f7-c9409e93e8ce | |
| DfMilkSilo | Succeeded | Data flow | 10/16/2026, 8:25:17 AM | 2m 8s | AutoResolveIntegrationRuntime (Central India) | | e5ae0414-e6b6-4048-b2f3-34390bb5e1e1 | |
| IfHasData | Succeeded | If Condition | 10/16/2026, 8:25:16 AM | Less than 1s | | | 7cdc1d09-9b5f-4ab4-a181-80c1c462877f | |
| FEMilkSilo | Succeeded | ForEach | 10/16/2026, 8:25:14 AM | Less than 1s | | | 6c747ac5-b569-4cf9-9f77-b67d46474a00 | |
| FilterMilkSilo | Succeeded | Filter | 10/16/2026, 8:25:14 AM | Less than 1s | | | b678190c-a391-46cb-872f-0fd22d817a14 | |
| GmMilkSilo | Succeeded | Get Metadata | 10/16/2026, 8:25:00 AM | 13s | AutoResolveIntegrationRuntime (Central India) | | bd09b6fe-a610-4070-a41c-d1b6d926306e | |
| SetMaxTimeMilkSilo | Succeeded | Set variable | 10/16/2026, 8:24:59 AM | Less than 1s | | | 2216c285-7c10-4b83-a892-d0a2495857b9 | |
| LkMilkSilo | Succeeded | Lookup | 10/16/2026, 8:24:38 AM | 20s | AutoResolveIntegrationRuntime (Central India) | | 917c4a2b-d1c1-4588-9bd3-8bcd40852b18 | |
| SetBatchEnd | Succeeded | Set variable | 10/16/2026, 8:24:37 AM | Less than 1s | | | 2be16833-6e72-470a-a87b-3c20843d9e5d | |

## 4.6 Mapping Data Flow

Show the Data Flow source, cleansing/derived-column transformations, validation rules, conditional split, deduplication logic, and Processed and Rejected sinks. Describe the rules actually implemented in the project.

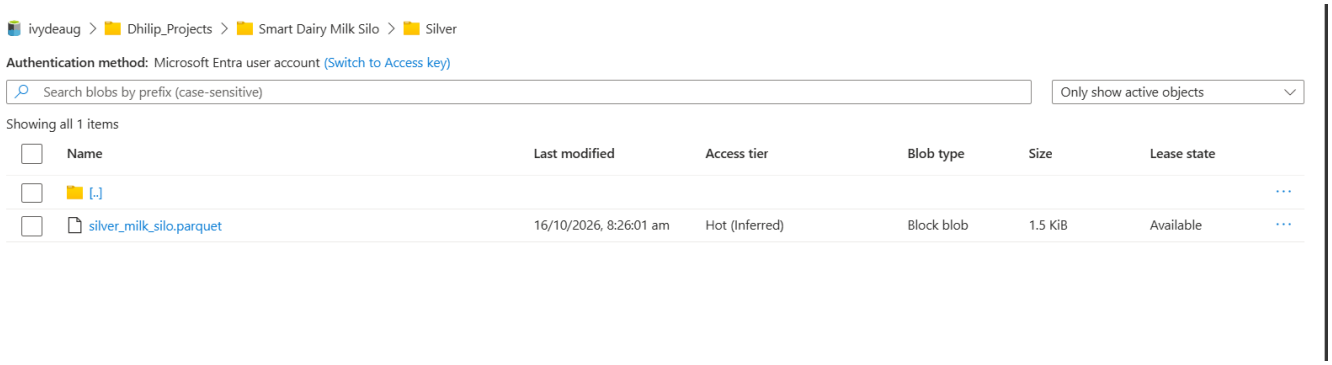
- Screenshot:



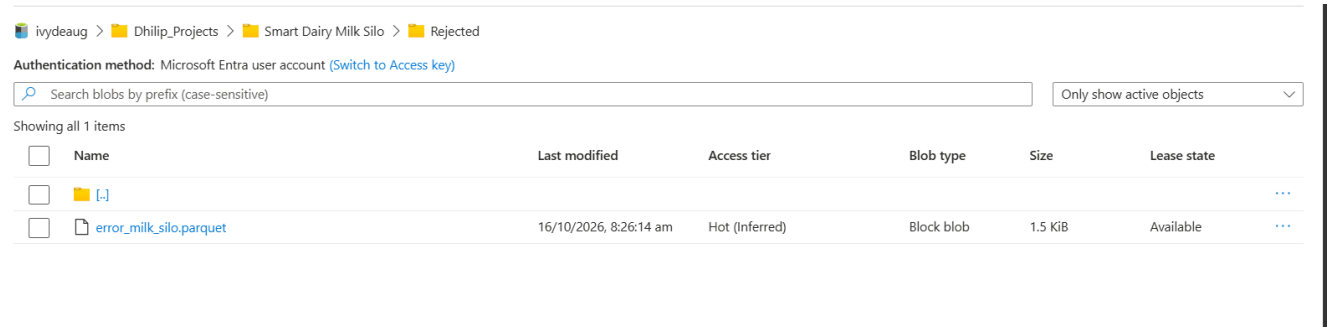

## 4.7 Processed and Rejected outputs

Open both output paths and show the generated files. The Processed path should contain records that pass the configured rules; the Rejected path should contain records that fail validation. If no invalid data exists, run a controlled test with a clearly invalid value and verify the rejection path.

- Screenshot:



Screenshot:




## 4.8 Azure SQL Database

Query the final processed-data table to show loaded rows. Query the watermark table to show the source identifier and last processed timestamp. Avoid exposing credentials or secrets.

Screenshot: Result for processed table

Data preview - DhilipMilkSilo_GoldFinalData — Showing 1000 rows — Search

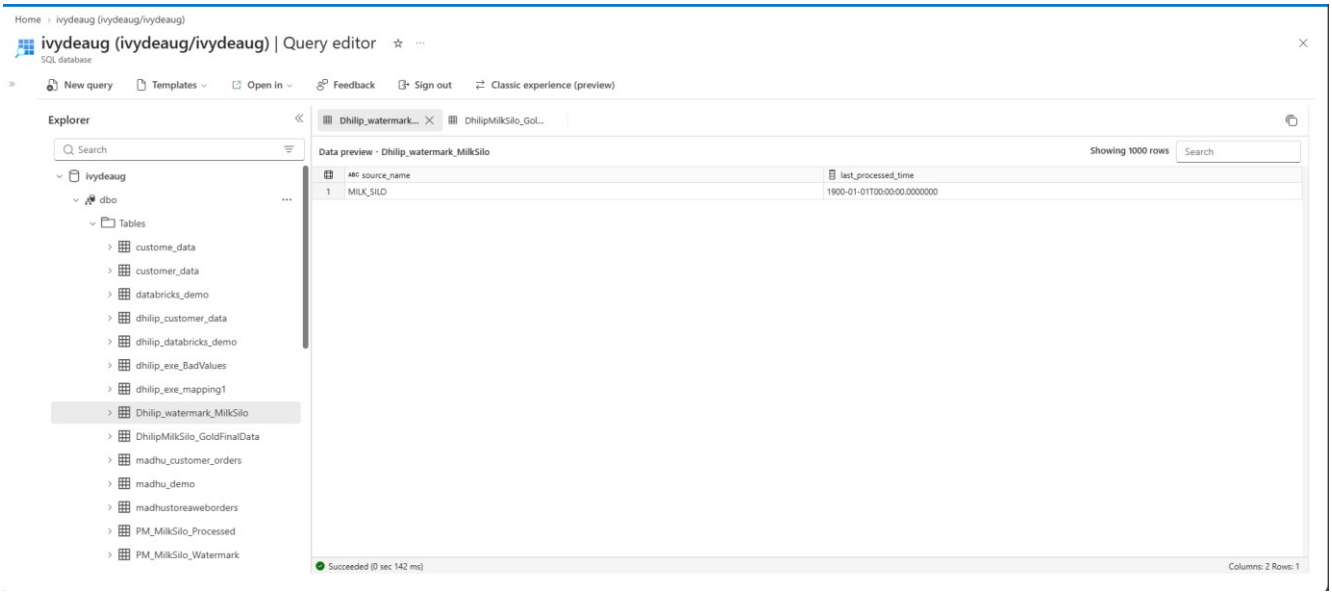
| | sample_id | sample_timesta... | silo_id | temperature_c | ph | density | fat_pct | freezing_point_c | mb_rt |
|---|---|---|---|---|---|---|---|---|---|
| 1 | SMP000101 | 2026-10-14T14:49:2... | SILO_04 | 3.20 | 6.68 | 1.0440 | 7.30 | -0.542 | 97 |
| 2 | SMP000103 | 2026-10-14T14:49:2... | SILO_03 | 4.40 | 6.65 | 1.0560 | 6.80 | -0.544 | 140 |
| 3 | SMP000104 | 2026-10-14T14:49:2... | SILO_05 | 2.20 | 6.52 | 1.0670 | 4.40 | -0.528 | 464 |
| 4 | SMP000105 | 2026-10-14T14:49:2... | SILO_05 | 3.00 | 6.50 | 1.0580 | 3.80 | -0.527 | 263 |
| 5 | SMP000106 | 2026-10-14T14:49:2... | SILO_04 | 5.40 | 6.50 | 1.0440 | 4.80 | -0.529 | 69 |
| 6 | SMP000107 | 2026-10-14T14:49:2... | SILO_02 | 3.00 | 6.66 | 1.0350 | 7.00 | -0.553 | 281 |
| 7 | SMP000108 | 2026-10-14T14:49:2... | SILO_05 | 5.60 | 6.63 | 1.0270 | 3.30 | -0.537 | 424 |
| 8 | SMP000109 | 2026-10-09T11:46:5... | SILO_03 | 3.20 | 6.67 | 1.0350 | 7.80 | -0.542 | 472 |
| 9 | SMP000111 | 2026-10-14T14:49:2... | SILO_01 | 6.00 | 6.55 | 1.0300 | 4.10 | -0.551 | 262 |
| 10 | SMP000112 | 2026-10-14T14:49:2... | SILO_05 | 3.20 | 6.51 | 1.0430 | 6.60 | -0.547 | 204 |
| 11 | SMP000114 | 2026-10-14T14:49:2... | SILO_05 | 2.20 | 6.63 | 1.0380 | 4.20 | -0.564 | 188 |
| 12 | SMP000115 | 2026-10-14T14:49:2... | SILO_02 | 3.50 | 6.51 | 1.0700 | 3.40 | -0.561 | 145 |
| 13 | SMP000118 | 2026-09-28T16:11:5... | SILO_05 | 4.10 | 6.54 | 1.0670 | 4.70 | -0.555 | 263 |
| 14 | SMP000120 | 2026-10-14T14:50:3... | SILO_01 | 6.00 | 6.63 | 1.0500 | 5.10 | -0.548 | 147 |
| 15 | SMP000121 | 2026-10-14T14:50:3... | SILO_03 | 5.00 | 6.67 | 1.0600 | 3.30 | -0.538 | 374 |
| 16 | SMP000122 | 2026-10-14T14:50:3... | SILO_03 | 4.90 | 6.73 | 1.0440 | 4.00 | -0.529 | 466 |

Succeeded (0 sec 189 ms) — Columns: 14 Rows: 303

Screenshot: SQL query and result for watermark table

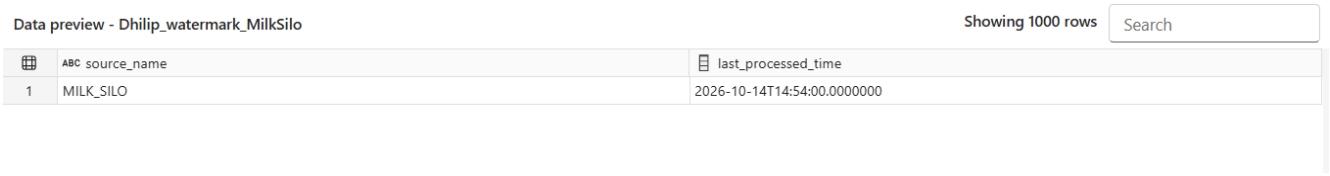
Run | Save as | Save as view | Download .sql

```
2
3   CREATE TABLE dbo.Dhilip_watermark_MilkSilo
4   (
5       source_name VARCHAR(50) NOT NULL PRIMARY KEY,
6       last_processed_time DATETIME2 NOT NULL
7           DEFAULT '1900-01-01T00:00:00'
8   );
9
10  INSERT INTO dbo.Dhilip_watermark_MilkSilo
11  (
12      source_name,
13      last_processed_time
14  )
15  VALUES
16  (
17      'MILK_SILO',
18      '1900-01-01T00:00:00'
19  );
20
21
22  SELECT *
23  FROM dbo.Dhilip_watermark_MilkSilo;
```




## 4.9 Incremental-load verification

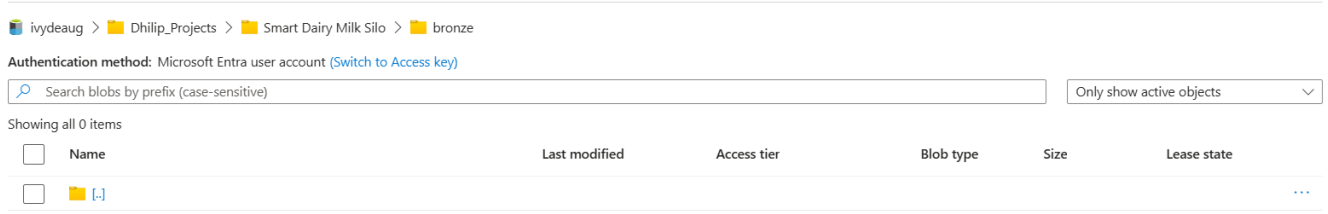
Record the current SQL row count and watermark. Run the pipeline again without adding new source data. Confirm from the results that existing records are not duplicated and that the watermark remains unchanged when there is no new eligible data. If the pipeline's design behaves differently, document the observed behavior accurately.




## 4.10 Cleanup verification

Show that the configured cleanup activity is dependent on successful completion of all required processing, SQL load, and watermark operations. Show the eligible source file before and after a successful run. Do not demonstrate destructive cleanup on the only copy of important evidence.

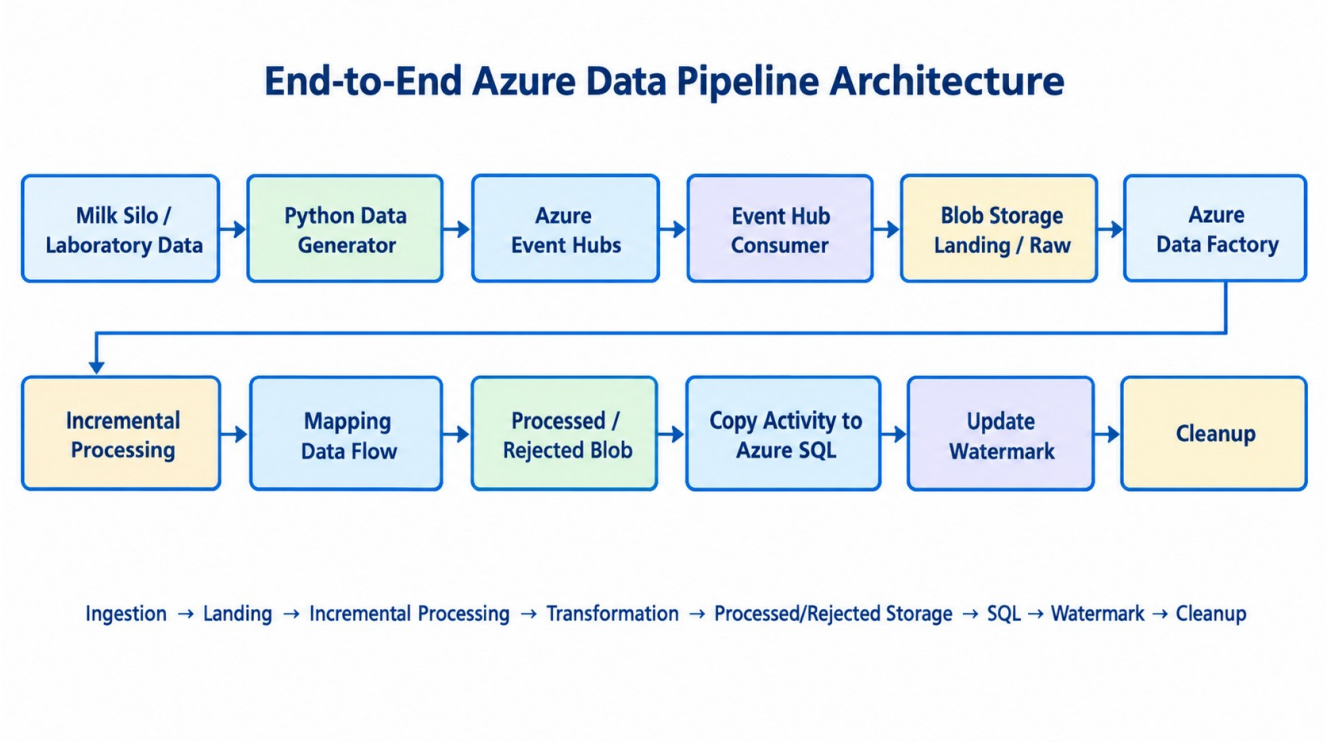
Screenshot: Delete/Cleanup activity and success dependency



## 4.11 Architecture summary

Finish by displaying the architecture diagram and summarizing ingestion, landing, incremental processing, transformation, processed/rejected storage, SQL loading, watermark update, and cleanup.

Screenshot: final architecture diagram






## 5. Conclusion

The demonstration is complete when the evidence shows the flow from event generation through Event Hubs, Blob Storage, ADF transformation, processed/rejected outputs, Azure SQL loading, watermark management, incremental rerun behavior, and cleanup. The final report should include actual screenshots and observed results so that each requirement can be independently verified.

*****